



Chapitre 2

Types, références, arithmétique

Structure du chapitre



- Variables
- Types de base
- Opérateurs
- Références
- Priorité des opérateurs
- Commentaires
- Entiers : représentation, limites, dépassement
- Réels : IEEE 754, précision
- Conversions entre types



1. Variables





- Les variables servent à **stocker des valeurs**.
- Une variable doit être **déclarée** avant son utilisation.
- Pour déclarer une variable, il faut
 - le **type de donnée** stocké (nombre entier, nombre réel, caractère, ...)
 - un **nom (identificateur)**
- Bonne pratique : **initialiser** les variables lors de la déclaration (création).
 - ... sans quoi son contenu est indéterminé

```
int mon_entier;
```

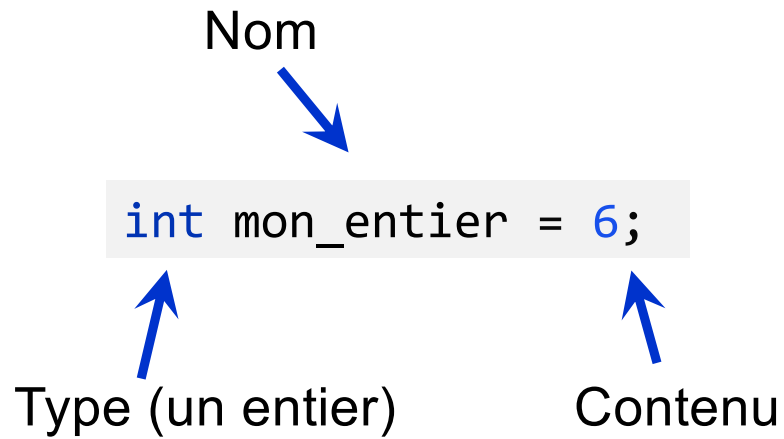
Déclaration, contenu indéterminé ☹

```
int mon_entier = 6;
```

Déclaration + initialisation ☺



Exemple de déclaration des variables



```
int nb_mois = 12;
```

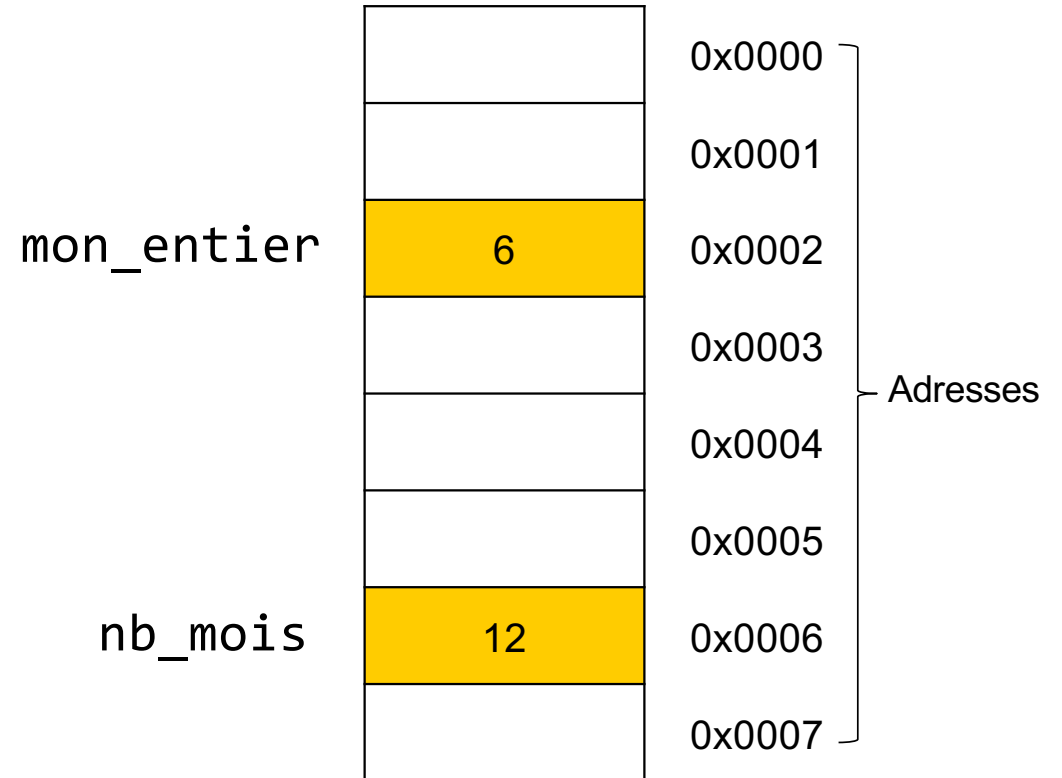


Illustration de la mémoire

Mais pourquoi utiliser des variables

- Faciliter la modification, la lecture et la réutilisabilité du programme.
- Stocker une entrée utilisateur, un résultat d'une opération, etc.

```
1 cout << "Nb bouteilles dans un pack : " << 6;  
2 cout << "Volume d'un pack(l) = " << 0.33 * 6;  
3 cout << "Poids d'un pack(g) = " << 13 * 6;  
4 cout << "Entrer le Nb de pack à expédier :";  
5 cout << "Le poids de votre colis(g) :" << 13 * 6 * nb_pack;
```

Dans le programme ci-dessus :

- Modifier le nombre (6) de bouteilles par pack nécessite de changer les lignes 1,2,3, et 5
- Le calcul du poids d'un pack (13.2 * 6) est répété aux lignes 3 et 5.
- Il faut une variable nb_pack pour stocker l'entrée utilisateur, ligne 4.

HE^{VD} IG Noms de variables



- Il est important de **choisir un nom explicite** qui **indique** à quoi sert cette variable

```
c = a * b;
```

est plus compréhensible avec des identificateurs parlants :

```
surface = largeur * hauteur;
```

- **Les règles syntaxiques C++** pour les noms d'identificateurs doivent être respectées



- [NL5](#) : Le nom d'une variable ne doit pas mentionner son type (i.e. n'utilisez pas la [notation hongroise](#) (e.g. nSize))
- [NL7](#) : La longueur d'un nom de variable doit être $\pm$ proportionnelle à sa portée (distance entre ses utilisations)
- [NL8](#) : Utilisez un style de nommage consistant : snake_case, [camelCase](#), PascalCase, ...
- [NL9](#) : N'utilisez pas TOUT_EN_MAJUSCULE pour les identificateurs autres que les macros (vues en ARO)
- [NL10](#) : Préférez le style snake_case, c'est le style utilisé par la Standard Template Library (STL). Utilisez éventuellement Majuscule_initiale pour les types que vous définissez vous-même, comme [Bjarne Stroustrup](#).



- Certaines variables ne doivent **pas changer de valeurs** après leurs initialisations dans le programme. On dit qu'elles sont **immutables**.
- Pour celles-ci, le C++ met à disposition le mot réservé **const** à placer juste avant la déclaration de la variable

```
const int vol_bouteille = 33;
```

- Son **initialisation** à la déclaration est **obligatoire**

Cpp core guideline [CON.1](#) : Par défaut, définissez toutes vos variables **const**. Les variables mutables (non **const**) devraient être l'exception

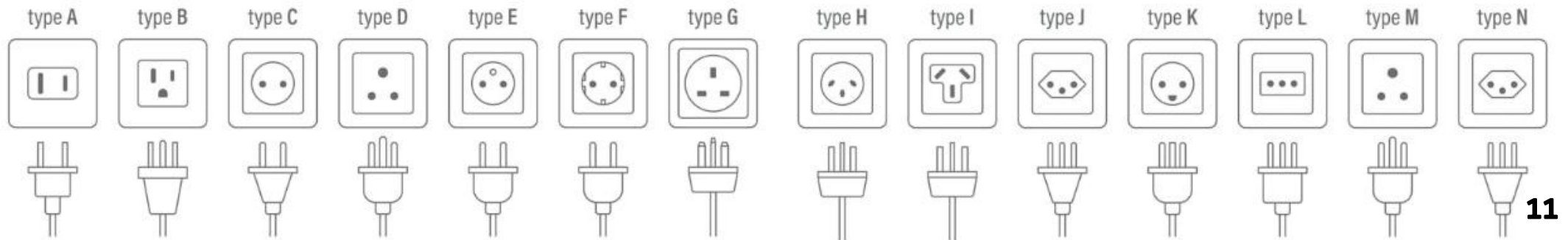
Pourquoi utiliser des constantes ?

- Reprenons notre exemple de pack de bouteilles. Le code est encore plus facile à comprendre et à maintenir en cas de changement

```
const int nb_bouteilles    = 6;
const int vol_bouteille    = 33;
const int poids_bouteille  = 13;
cout << "volume d'un pack(c1)      = " << vol_bouteille * nb_bouteilles;
cout << "Nb bouteilles dans un pack = " << nb_bouteilles;
const int poids_pack = poids_bouteille * nb_bouteilles;
cout << "poids d'un pack(g)         = " << poids_pack;
cout << "entrer le nb de pack à expédier :";
int nb_pack; // seule variable mutable
cin >> nb_pack;
cout << "le poids de votre colis(g) : " << poids_pack * nb_pack;
```

- On évite aussi l'apparition d'un « **magic number** » inexpliqué au milieu du code

2. Types de base





- Les données en C++ sont **typées**
- Un **type** définit
 - comment cette donnée est **stockée en mémoire** (nombre de bits et codage)
 - les **opérations** possibles avec cette donnée
- Les types **fondamentaux** (ou de base) fournis par le langage permettent de stocker des données simples
 - **caractères** typographiques
 - nombres **entiers**
 - nombres **réels**
 - **booléens**

HE^{VD} IG Les types de base



```
int nb_packs = 10;
```

- Stockent une valeur entière comme -7 ou 1024
- Il peut être qualifié par la **taille** utilisée pour le stocker et le fait d'être **signé** ou pas (*voir la section Entiers*)

```
float volume_canette = 0.33f;  
double pi = 3.141592;
```

- Stockent des nombres réels tels que 3.14 ou 0.01
- Le type **double** permet plus de précision que **float**.

HE^{VD} IG Les types de base



```
char c = 'a';
```

- Le type `char` permet de stocker un **caractère**.
- En réalité il stocke un **entier**, sa taille est de 1 byte.
- Le code **ASCII** fait correspondre caractères et valeurs numériques.
- Pour définir une valeur littérale de type `char` on utilise les **simples guillemets**.

HE^{VD} IG Les types de base



```
string nom      = "Bob";  
string prenom   = "Alice";  
string titre    = "Programmation en c++";  
string ville;
```

- Le type `string` permet de stocker une **chaîne de caractères**.
- Contrairement aux types simples une variable **non initialisée** est définie comme une **chaîne vide**.
- Pour définir une variable de type `string` on utilise les **doubles guillemets**.

HE^{VD} IG Les types de base



```
bool vrai = true;  
bool faux = false;
```

- Stockent une valeur booléenne : vraie ou fausse (`true` / `false`)
- C'est en fait un type numérique qui peut prendre les valeurs
 - 0 pour `false`
 - 1 pour `true`
- Toute valeur numérique non nulle correspond à `true`.

```
bool vrai = 42;  
bool faux = 0;
```



3. Opérations





- En C++, tout ce qui correspond à une valeur est une **expression**
- On dit qu'elle « **renvoie** » une valeur

```
int valeur = 3; // déclaration  
  
4           // renvoie la valeur 4  
valeur      // renvoie la valeur 3  
5 + 2       // renvoie la valeur 7
```

- Nous en avons déjà vu plusieurs sortes
 - Les littéraux constants
 - L'opérateur d'affectation

HE^{VD} IG Littéraux constants



- La plus simple des expressions est une constante exprimée littéralement
 - un caractère : 'A' entre **simples** guillemets
 - une chaîne de caractères : "Hello, World!" entre **doubles** guillemets
 - un entier : 59
 - un réel : 3.1416

Pour aller plus loin : lvalue vs rvalue

- **lvalue** (valeur de localisation) :
 - est une expression qui fait **référence à une adresse mémoire**.
 - une lvalue **a une adresse** qui peut être **accessible** par le programme.
 - Exemple : les noms de variables
- **rvalue** (valeur de résultat) :
 - est une expression qui **ne fait pas référence à une adresse mémoire** (n'est pas un lvalue).
 - Exemple : les littéraux constants, les résultats d'expressions qui ne se résolvent pas en lvalues

lvalue → *rvalue*

```
int x = 7;
```

Opérateur d'affectation =

- L'opérateur d'affectation = copie la valeur de l'expression à droite dans la variable à gauche
- Exemple: `age = 10;` affecte la valeur 10 à la variable age
- Elle remplace la valeur actuelle de la variable par une nouvelle valeur ce qui écrase la valeur qui s'y trouvait précédemment

`10 = age;` **X**



- Les types réels (`float`, `double`, ...) disposent des opérateurs `+`, `-`, `*`, et `/` qui se comportent comme en mathématiques

```
double somme = 1.0 + 3.0;           // 4.0  
  
double difference = 7.0 - 3.0;      // 4.0  
  
double produit = 3.14 * 2.0;        // 6.28  
  
double quotient = 5.0 / 2.0;        // 2.5
```



- Les types entiers (`int`, ...) disposent des opérateurs `+`, `-`, `*`, `/`, et `%`
- Les opérateurs `+`, `-`, et `*` se comportent comme pour les réels
- L'opérateur `/` effectue une **division entière** et donne donc un résultat entier
- L'opérateur `%` calcule le reste d'une division entière, appelé **modulo**

```
int somme = 1 + 3;  
// 4  
  
int difference = 7 - 3;  
// 4  
  
int produit = 3 * 2;  
// 6  
  
int quotient_entier = 5 / 2;  
// 2, pas 2.5  
  
int modulo = 8 % 3;  
// 2, le reste de la division  
// entière de 8 par 3
```

Division entière et modulo

- Pour a et b entiers, on a toujours l'égalité suivante :

$$(a/b) * b + (a\%b) == a$$

- Pour a et/ou b négatifs, il suffit de se souvenir que $(a \% b)$ est du même signe que a

```
int a = 7, b = 4;    // ou -7, ou -4
cout << "a = " << a << ", ";
cout << "b = " << b << ", ";
cout << "a/b = " << a/b << ", ";
cout << "a%b = " << a%b << endl;
```

a =	7,	b =	4,	a/b =	1,	a%b =	3
a =	-7,	b =	4,	a/b =	-1,	a%b =	-3
a =	7,	b =	-4,	a/b =	-1,	a%b =	3
a =	-7,	b =	-4,	a/b =	1,	a%b =	-3



- Combien font 1729 centimes en francs ?

```
int total = 1729; // centimes
int francs = total / 100;
int centimes = total % 100;

cout << total << " centimes valent " << francs
     << " francs et " << centimes << " centimes";
```

1729 centimes valent 17 francs et 29 centimes

Opérateurs d'affectation composée

- Il est courant de modifier une variable pour lui ajouter ou soustraire une valeur, la multiplier par une valeur, etc.
- Les opérateurs `+=`, `-=`, `*=`, `/=`, et `%=` permettent de combiner opération arithmétique et affectation

Expression	Affectation composée équivalente
<code>nbre = nbre + 2;</code>	<code>nbre += 2;</code>
<code>total = total - rabais;</code>	<code>total -= rabais;</code>
<code>bonus = bonus * 2;</code>	<code>bonus *= 2;</code>
<code>prix = prix / 2;</code>	<code>prix /= 2;</code>
<code>taux = taux % 100;</code>	<code>taux %= 100;</code>



Incrémentation / décrémentation

- Incrémenter et décrémenter de 1 est une opération si fréquente qu'il y a en C++ des opérateurs qui y sont dédiés : `++` et `--`

```
int compteur = 0;

compteur = compteur + 1; // vaut 1
compteur += 1;           // vaut 2
compteur -= 1;           // vaut 1
compteur++;              // vaut 2
compteur--;              // vaut 1
```



Opérateurs de comparaison

- C++ fournit 6 opérateurs de comparaison
 - < plus petit que
 - > plus grand que
 - <= plus petit ou égal
 - >= plus grand ou égal
 - == égal (ne pas confondre avec l'affectation =)
 - != différent de
- Ils permettent de comparer des
 - entiers
 - réels
 - chaînes de caractères (string)
- Ils renvoient des valeurs de type **bool**

HE^{VD} IG Opérateurs logiques



- C++ dispose des opérateurs logiques suivants

- not
- or
- and

not « ! »	
0	1
1	0

or « »		
0	0	0
0	1	1
1	0	1
1	1	1

and « && »		
0	0	0
0	1	0
1	0	0
1	1	1

```
cout << ( (5 == 5) && (3 > 6) ) << endl; // 0 (false)
cout << ( (5 == 5) || (3 > 6) ) << endl; // 1 (true)
cout << ( (5 == 5) and (3 < 6) ) << endl; // 1 (true)
cout << ( (5 != 5) or (3 > 6) ) << endl; // 0 (false)
cout << !(5 == 5) << endl; // 0 (false)
cout << not(5 != 5) << endl; // 1 (true)
```

Construire une condition sûre



- Dans une condition and / or, l'évaluation se fait de gauche à droite et s'arrête dès que le résultat est connu (évaluation court-circuit).
- On en profite pour écrire des conditions sûres : tester d'abord la validité, ensuite utiliser la valeur.
 - ici, la division m / n n'est calculée que si n n'est pas nul

```
((n != 0) and (p < m / n))
```

- Règle pratique : dans une condition, placez d'abord le test qui protège ($n \neq 0$), ensuite l'expression protégée (m / n).

HE^{VD} IG Lois de De Morgan



- La négation de la conjonction de deux propositions est équivalente à la disjonction des négations des deux propositions

$\neg (A \text{ and } B)$ est équivalent à $(\neg A \text{ or } \neg B)$

- La négation de la disjonction de deux propositions est équivalente à la conjonction des négations des deux propositions

$\neg (A \text{ or } B)$ est équivalent à $(\neg A \text{ and } \neg B)$



- Comme le type char stocke un entier correspondant au code ASCII du caractère, nous pouvons l'utiliser avec les opérations arithmétiques.

```
char x = 'B'; // 66
cout << x; // affiche B

x += 1;
cout << x; // affiche C

x += 32;
cout << x; // affiche c

x = x - 1;
cout << x; // affiche b
```



4. Références



- Une référence donne un second nom (un synonyme) à une variable existante.

```
int a = 0;
int& ref_a = a;
ref_a = 42;
    // change la valeur de a,
    // pas la variable référencée
cout << a;
    // affiche 42
```

- **Déclarer** une référence utilise le symbole **&** après le type. Elle **doit** être **initialisée**. La référence se comporte comme un synonyme de la lvalue référencée. Il n'y a pas de copie effectuée
- Une référence **reste liée à sa variable** : l'affectation modifie la **valeur**, jamais le **lien**. Elle ne peut donc pas désigner une constante.



- Une référence constante permet de lire une variable sans pouvoir la modifier.

```
int a = 42;  
const int& cref_a = a;  
cout << cref_a;  
    // affiche 42  
cref_a = 0;  
    // ne compile pas
```

- Déclarer une référence constante utilise le symbole & après le **type** et le mot clé **const**. Elle doit être initialisée.
- On ne peut changer ni l'emplacement mémoire référencé ni la valeur qui y est stockée.

5. Priorités des opérateurs



Opérateurs - priorités

- Comme en mathématiques, les **opérateurs multiplicatifs** (*, / et %) **ont priorité** sur les opérateurs additifs (+ et -)
 - $2 * 3 + 4$ vaut 10, et pas 14
- Si vous voulez changer l'ordre des calculs, il suffit d'ajouter des **parenthèses**
 - $2 * (3 + 4)$ vaut 14
- Quand deux ou plusieurs opérateurs arithmétiques ont la même priorité, ils sont appliqués de gauche à droite ou de droite à gauche dans l'expression selon leurs règles d'association.

https://en.cppreference.com/w/cpp/language/operator_precedence

Que valent les expressions suivantes ?

Expressions	Résultats
<code>2 + 3 * 3</code>	11
<code>10 - 10 / 3</code>	7
<code>10 % 3 + 1</code>	2
<code>4 + 2 * 2 * 5 - 1</code>	23
<code>(4 + 2) * 2 * (5 - 1)</code>	48
<code>a = 1; 9 / 3 + ++a</code>	5
<code>(3.2 + 0.6) * -2.0</code>	-7.6
<code>5 == 5 && 2 > 5 (3 <=> 3 == 0)</code>	true



6. Commentaires

```
// dernière section du chapitre  
// il ne reste que deux slides  
// accrochez-vous ...
```



- Permettent d'inclure des **explications** en langage courant, données **à la personne** qui lit le code
- Ils sont **ignorés** par le **compilateur**

```
double volumeCanette = 0.33; // en Litre
```



- Éviter de surcharger le code de commentaires redondants et/ou d'explications inutiles. Le choix des **noms** de vos variables et fonctions et la **structure** de votre code est au moins aussi importante que vos commentaires pour sa compréhension

Commentaires – 2 styles

- après `//` et jusqu'à la fin de la ligne

```
// Commentaire sur une ligne
```

```
double volume = 2; // il peut suivre du code
```

```
// On peut écrire sur plusieurs lignes
```

```
// en les commençant toutes par deux fois le symbole /
```

- entre `/*` et `*/`. Il peut commencer et s'arrêter en cours de ligne, ou en occuper plusieurs

```
/* ceci est un commentaire sur plusieurs lignes qui commence  
par slash-étoile et fini par étoile-slash */
```

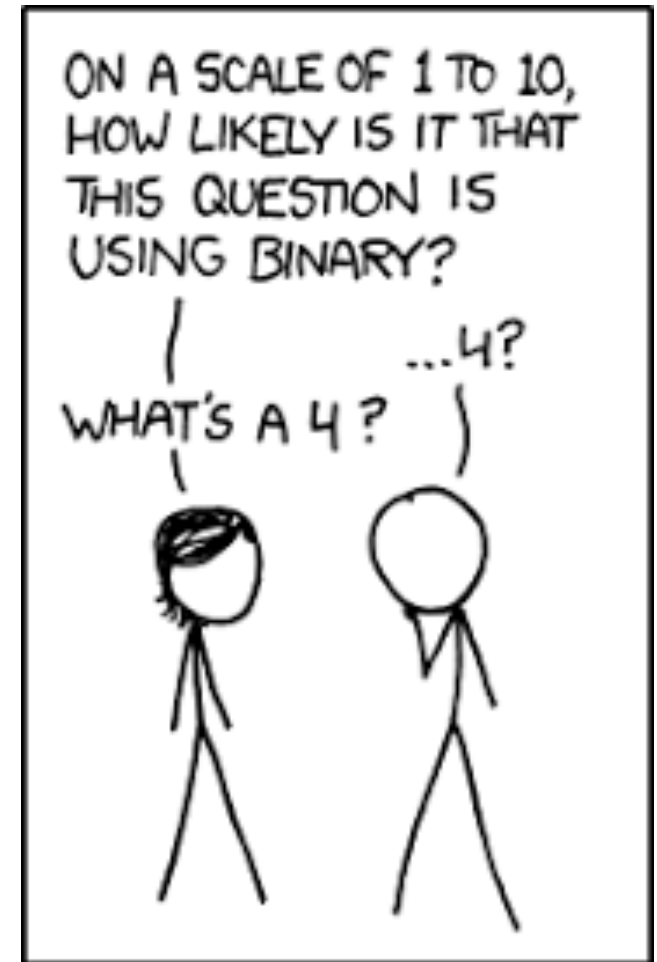
```
/* mais rien n'empêche de n'écrire qu'une ligne */
```

```
double /* n'importe */ volume /* où */ = 2;
```

7. Entiers



- Entiers signés : représentation en mémoire, complément à 2
- `sizeof`, `numeric_limits`
- Dépassement : le détecter, le prévenir
- Entiers non signés, `std::size_t`
- Entiers littéraux



<https://xkcd.com/953/>

Entiers signés et non signés

- Un entier **signé** peut être négatif, nul ou positif ; un entier **non signé** ne code que des valeurs ≥ 0
- Sur n bits, les deux ont le même nombre de **combinaisons** possibles (2^n) : seule leur **interprétation** change
 - en signé, la moitié code des négatifs ; en non signé, le maximum est $\sim 2 \times$ plus grand
- Une combinaison de bits n'a pas de signe en soi : c'est le **type déclaré** qui dit comment le lire
- En C++, les entiers sont signés par défaut. Les non signés sont vus plus loin.

Combinaison (8 bits)	Lu comme signé	Lu comme non signé
0000 0000	0	0
0111 1111	127	127
1000 0000	-128	128
1111 1111	-1	255

HE^{VD} IG Entiers signés



- Stockés en mémoire sous forme binaire
 - **Bit** = chiffre binaire : 0 ou 1
 - **Octet** = groupe de 8 bits
 - **Byte** = plus petit groupe de bits adressable. Typiquement un octet, mais pas toujours. Le nombre exact est défini dans la constante `CHAR_BIT` provenant de `<climits>`.
- Le **nombre de bits utilisés** détermine le nombre d'entiers représentables
 - Avec n bits - $\frac{n}{8}$ octets - on peut représenter jusqu'à 2^n entiers différents
 - Les entiers positifs de 0 à $2^{n-1} - 1$ sont codés en base 2. Leur premier bit est toujours 0
 - Pour les entiers négatifs, cela dépend de leur représentation, qui n'est pas spécifiée par le standard C++. Leur premier bit est toujours 1.



- Même si ce n'est pas obligatoire, tous les systèmes actuels utilisent le complément à 2 pour représenter les entiers négatifs
 - Coder la valeur absolue en base 2
 - Inverser tous les bits (0 devient 1, 1 devient 0)
 - Ajouter 1
- En complément à 2 avec des entiers sur n bits, on peut coder les entiers négatifs de -2^{n-1} à -1

*Note : les autres représentations diffèrent peu. Par exemple, une représentation signe + amplitude peut coder les entiers de $-(2^{n-1} - 1)$ à $2^{n-1} - 1$.
L'entier 0 utilise alors 2 représentations binaires*

Types entiers signés en C++

- Le C++ propose 4 tailles d'entiers signés

```
signed short int  
signed int  
signed long int  
signed long long int
```

- La nombre de bits utilisés par ces types n'est pas garanti. Les seules garanties sont
 - `sizeof(short) <= sizeof(int) <= sizeof(long) <= sizeof(long long)`.
 - `short` et `int` utilisent au moins 16 bits, `long` au moins 32, et `long long` au moins 64
- On peut également utiliser le type caractère `signed char` comme un entier sur 8 bits. Le mot clé `signed` n'est pas optionnel dans ce cas



- L'opérateur `sizeof` retourne le nombre de **bytes** utilisés par le type ou l'expression en paramètre

- Syntaxe :

```
sizeof(type)
sizeof expression
```

- Utilisation :

```
cout << "s char      : " << sizeof(signed char) << endl;
cout << "short       : " << sizeof(short)         << endl;
cout << "int         : " << sizeof(int)           << endl;
cout << "long        : " << sizeof(long)          << endl;
cout << "long long   : " << sizeof(long long)      << endl;
short int a = 42;
cout << "a          : " << sizeof a               << endl;
```

s char	:	1
short	:	2
int	:	4
long	:	8
long long	:	8
a	:	2



- La manière la plus propre de connaître les propriétés d'un type numérique est d'utiliser les méthodes de `std::numeric_limits` , provenant de la librairie `<limits>`
- Syntaxe :

```
std::numeric_limits<type>::lowest()    // plus petite valeur représentable
std::numeric_limits<type>::max()        // plus grande valeur représentable
std::numeric_limits<type>::digits      // nombre de bits hors bit de signe
std::numeric_limits<type>::is_signed   // vrai pour les types signés
std::numeric_limits<type>::is_integer  // vrai pour les types entiers
```



- Le résultat d'un calcul effectué avec un type donné peut ne pas être représentable dans ce type. On parle de dépassement
- Exemple avec `std::numeric_limits<int>::max()` qui vaut 2'147'483'647 :

```
int a = 2'000'000'000;  
int b = 1'000'000'000;  
int c = a + b;  
cout << a << " + " << b << " = "  
      << c << endl;
```

2000000000 + 1000000000 = -1294967296

- Selon le standard C++, un dépassement sur un calcul en type entier signé donne un résultat **non défini**, et donc inutilisable

Prévenir un dépassement



- On peut vérifier si un calcul va dépasser avant de l'effectuer
- L'addition déborde si
 - Les 2 opérandes sont positives et le résultat est $> \text{numeric_limits}<\text{int}>::\text{max}()$
 - Les 2 opérandes sont négatives et le résultat est $< \text{numeric_limits}<\text{int}>::\text{lowest}()$
- Le code suivant vérifie ces deux possibilités

```
bool sum_would_overflow(int lhs, int rhs) {  
    return (lhs >= 0) ?  
        numeric_limits<int>::max() - lhs < rhs :  
        rhs < numeric_limits<int>::lowest() - lhs;  
}
```

- Soustraction et multiplication peuvent également déborder

HE^{VD} IG Vol 501 d'Ariane 5



- Vol inaugural, 4 juin 1996
- Même système de guidage inertiel qu'Ariane 4
- Accélérations 5 fois plus fortes qu'Ariane 4
- Dépassement dans le calcul de la position à partir des accélérations
- L'ordinateur de bord ordonne un virage serré pour corriger la trajectoire
- L'accélération latérale arrache un des boosters latéraux
- La destruction automatique est engagée ☹️



HE^{VD} IG Entiers non signés



- A chacun des types d'entier signé correspond un type d'entier non signé de même taille en remplaçant le mot clé `signed` par `unsigned`

```
unsigned char  
unsigned short int  
unsigned int  
unsigned long int  
unsigned long long int
```

- Un type `unsigned` de n bits peut représenter les entiers positifs de 0 à $2^n - 1$
- En mémoire, les bits correspondent à la représentation de l'entier en base 2

HE^{VD} IG Arithmétique modulo 2^n



- Il n'y a **pas de dépassement** dans les calculs effectués en unsigned
- Les opérateurs +, -, et * sont définis « modulo 2^n »

```
unsigned a32 = 2'000'000'000, b32 = 3'000'000'000;  
unsigned long long a64 = a32, b64 = b32;  
cout << a32 << " + " << b32 << " = " << a32 + b32  
      << " = " << a64 + b64 << " mod 4294967296";
```

```
2000000000 + 3000000000 = 705032704 = 5000000000 mod 4294967296
```

- En pratique, aucun calcul de modulo n'est effectué par le processeur. Il se contente d'ignorer les bits de report au-delà du $n^{\text{ième}}$ bit lors des calculs.
- Le unsigned le plus grand et 0 sont des entiers consécutifs

```
unsigned zero = 0; cout << zero - 1 << endl;  
cout << numeric_limits<unsigned>::max() + 1 << endl;
```

```
4294967295  
0
```

HE^{VD} IG `std::size_t`



- Type de retour de l'opérateur `sizeof`, `string::size()`, ...
- Alias d'un type **entier non signé** suffisamment grand pour **stocker la taille** de tout objet en C++ y compris des tableaux
- Défini dans de nombreux headers : `<cstddef>`, `<cstdio>`, `<cstdlib>`, ...
- Peut-être synonyme de `unsigned int` (e.g. win32), `unsigned long` (e.g. macOS) ou `unsigned long long` (e.g. win64)
- Type des **indices** pour accéder aux éléments d'un `std::array`, d'un `std::vector`, ou aux caractères d'une `std::string`

HE^{VD} IG Entiers littéraux



- Les entiers littéraux permettent d'écrire une valeur entière dans le code.
- Le séparateur ' facilite la lecture des grands nombres.
- Pour aller plus loin : des préfixes (base 2, 8, 16) existent - voir `cppreference`.

```
int un_million = 1'000'000;
```

HE^{VD} IG Suffixes de type



- Les entiers littéraux peuvent être suivis d'un suffixe qui spécifie le type

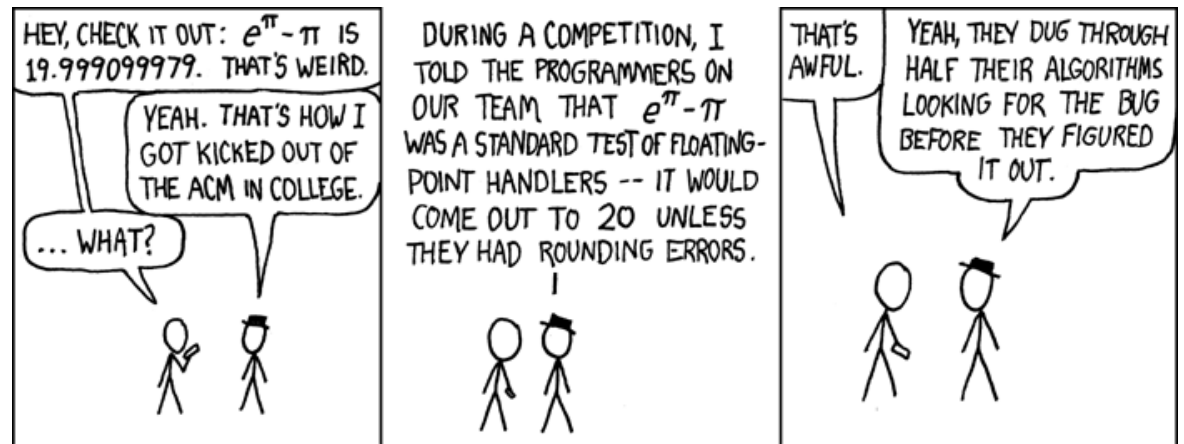
Pas de suffixe	<code>signed int</code>
L	<code>signed long int</code>
LL	<code>signed long long int</code>
U	<code>unsigned int</code>
UL	<code>unsigned long int</code>
ULL	<code>unsigned long long int</code>

- Minuscule ou majuscule à choix

8. Réels



- Représentation en mémoire : IEEE 754, NaN
- Types réels en C++ : float, double
- Égalité vs presque égalité
- Réels littéraux, setprecision
- <cmath>





- Le concept de **virgule flottante** permet de représenter de **manière approchée** une large plage de nombres réels
- Il consiste à représenter le réel r sous la forme $r = (-1)^s \cdot m \cdot b^e$, avec
 - b la **base** entière, typiquement 2 ou 10
 - s le **signe** binaire $\in \{0,1\}$
 - e l'**exposant** entier
 - m la **mantisse** réelle, avec $1 \leq m < b$ dans sa forme normalisée pour qu'une seule paire (m, e) code le même r .
- Par exemple, le réel $r = 314,2$ peut être représenté comme
 - $r = + 3,142 \cdot 10^2$ en base $b = 10$
 - $r = + 1,22734375 \cdot 2^8$ en base $b = 2$

IEEE 754 : IEEE Standard for Floating-Point Arithmetic

- Etabli et mis à jour depuis 1985 par l'IEEE : Institute of Electrical and Electronics Engineers
- Définit les standards d'arithmétique en virgule flottante binary16, binary32, binary64, binary128, binary256, decimal32, decimal64, et decimal128
- Chacun de ces standards spécifie notamment
 - La base b : 2 ou 10
 - Le nombre de bits utilisés en tout : 16, 32, 64, 128, ou 256
 - La précision p de chiffres en base b utilisés par la mantisse entière M
 - La manière de coder les chiffres décimaux en bits pour les standards decimal...
 - Le biais B et donc la plage d'exposants e utilisables avec les bits restants



- Enfin, le plus grand exposant e et $M \neq 0$ codent « **not a number** », qui correspond au résultat de calculs impossibles ou indéterminés en nombres réels

```
cout << 0./0. << endl;  
cout << sqrt(-1) << endl;  
cout << 1./0. - 1./0. << endl;
```

```
nan  
nan  
nan
```

- Les **comparaisons** impliquant un NaN sont particulières, NaN n'étant pas ordonné par rapport aux autres réels
 - $<$, $>$, $<=$, $>=$ et $==$ retournent toujours **false** dès qu'au moins un opérande est NaN.
 - $!=$ retourne toujours **true**.
- Pour savoir si une valeur est un NaN, on utilise donc `isnan(x)`, de `<cmath>`



- C++ supporte trois longueurs de nombres réels via les 3 types suivants
 - float simple précision
 - double double précision
 - long double précision étendue
- float et double correspondent aux formats IEEE-754 binary32 et binary64 s'ils sont supportés par l'architecture du processeur

Type et nombre de bits		Signe	Mantisse	Exposant
float	32	1	23	8
double	64	1	52	11

- long double peut utiliser 64, 80 ou 128 bits selon la mise en œuvre

Egalité vs. « presque égalité »

```
int main() {  
    double d1 = 1. / 5.;  
    double d2 = 1. / sqrt(5.) / sqrt(5.);  
  
    // afficher 20 chiffres après virgule  
    cout << fixed << setprecision(20)  
        << "d1=" << d1 << '\n'  
        << "d2=" << d2 << '\n';  
  
    cout << ((d1 == d2) ? "d1 == d2" : "d1 != d2") << endl;  
}
```

```
d1=0.2000000000000000001110  
d2=0.199999999999999998335  
d1 != d2
```

Deux calculs mathématiquement équivalents peuvent produire des arrondis différents. **== exige une égalité exacte des valeurs stockées, pas des valeurs mathématiques attendues** : ne jamais comparer deux réels avec ==.



- Plutôt que l'égalité, on teste si l'**écart** entre les deux valeurs est plus petit qu'une **tolérance**
 - `fabs(v)` : valeur absolue d'un réel, de `<cmath>`
 - `fabs(x - y)` est donc la distance entre `x` et `y`

```
// tolérance absolue : quand on connaît l'ordre de grandeur attendu  
if (fabs(x - y) < 1e-9) ...
```

```
// tolérance relative : quand il varie  
if (fabs(x - y) <= 1e-9 * max(fabs(x), fabs(y))) ...
```

- **Absolue** : écart maximal fixe. Valable seulement autour d'un ordre de grandeur connu
- **Relative** : l'écart autorisé est une **fraction** des valeurs comparées. `1e-9` signifie « les 9 premiers chiffres significatifs coïncident », à toute échelle
- Cas de 0. Seule l'absolue fonctionne : une fraction de 0 vaut 0

Une version rigoureuse raisonne en ULP, l'écart entre deux réels voisins, et traite à part les nombres dénormalisés proches de zéro. Voir `almost_equal` sur `cppreference`.

HE^{VD} IG `setprecision(n)`



- Précise le nombre de chiffres décimaux à afficher
 - après la virgule en `fixed` et `scientific`
 - avant et après la virgule en `defaultfloat`, sans compter les zéros initiaux, et en n'affichant pas les zéros finaux après la virgule si `noshowpoint`
- Le dernier chiffre est arrondi au plus proche
- Des chiffres supplémentaires peuvent apparaître quand la précision demandée est plus grande que celle du type utilisé. Ils correspondent à la conversion en décimal de la représentation binaire.

```
float pi10 = 31.415926;  
cout << cout.precision() << endl  
    << scientific << pi10 << endl  
    << fixed << pi10 << endl  
    << defaultfloat << pi10 << endl  
    << setprecision(4) << pi10 << endl  
    << setprecision(30) << pi10 << endl  
    << showpoint << pi10 << endl;
```

```
6  
3.141593e+01  
31.415926  
31.4159  
31.42  
31.4159259796142578125  
31.4159259796142578125000000000
```

HE^{VD} IG Réels littéraux



- Tous les formats d'affichage peuvent être utilisés pour des constantes littérales réelles.
- Le type par défaut est double.
- Les suffixes F ou f (resp. L ou l) spécifient une constante littérale de type float (resp. long double)
- Attention, il est nécessaire qu'au moins un élément permette de savoir qu'il ne s'agit pas d'un entier littéral : . / e
- Un réel littéral qui déborde vaut l'infini

```
auto r1 = 2.;           // double
auto r2 = 2e+1;         // double
auto r3 = -0x1.p2f;     // float
auto r4 = 3.14L;        // long double
auto r5 = 0x2p2;        // double
auto r6 = 3.1E-4L;      // long double

auto e1 = 3;            // int
auto e2 = 314L;         // long int
auto e3 = 0x2f;         // int (47)
// auto nc1 = 314F ne compile pas

auto r7 = 1e600000;     // double (inf)
auto r8 = -1e600000;    // double (-inf)
```

HE^{VD} IG `#include <cmath>`



`<cmath>` fournit les fonctions

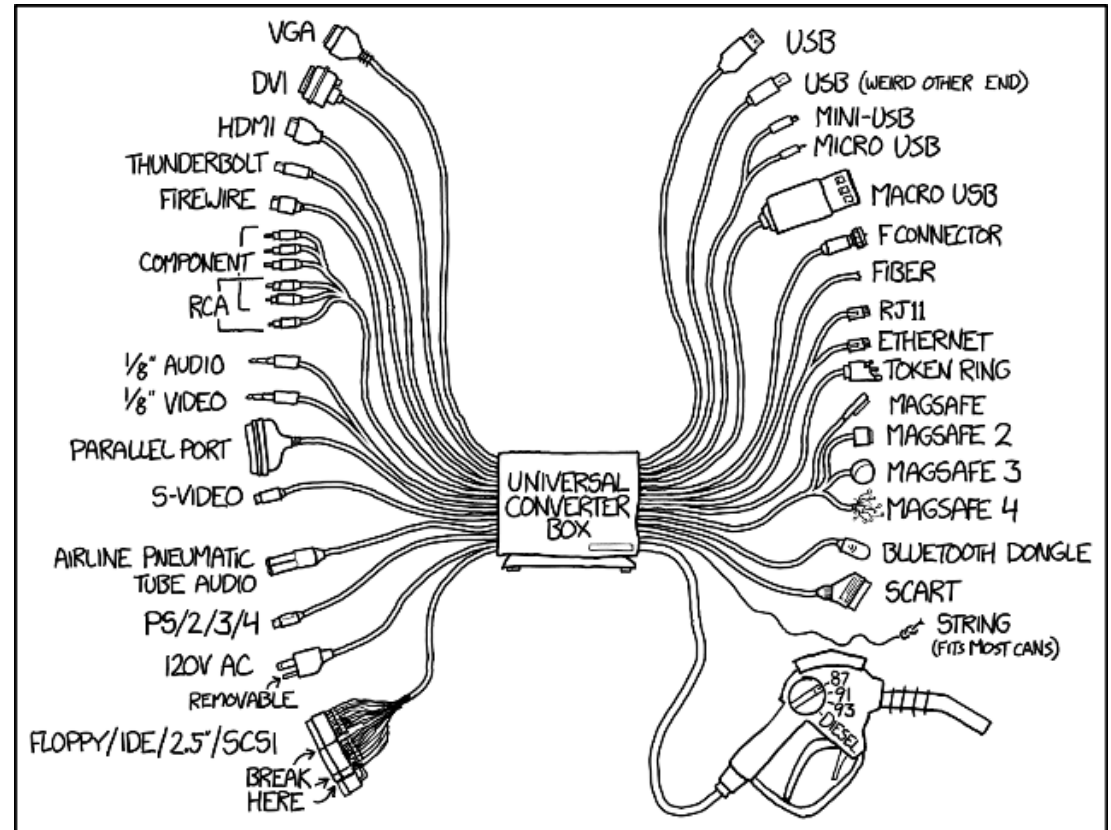
- Trigonométriques sin, cos, tan, asin, acos, ...
- Hyperboliques cosh, sinh, tanh, asinh, ...
- Exponentielles et logarithmiques exp, log, ...
- Puissances pow, sqrt, ...
- Valeur absolue abs, fabs, ...
- Arrondi round, ceil, floor, ...
- ...

Voir <http://www.cplusplus.com/reference/cmath/>

9. Conversions entre types



- Syntaxe
- Types de conversions numériques
 - Promotion numérique entière
 - Conversions entier → entier, ? → réel, et réel → entier
 - Arrondis
- Conversions implicites



<https://xkcd.com/1406/>

Syntaxe de conversion

- Pour convertir explicitement une expression d'un type à un autre, ce cours utilise une seule écriture : `static_cast`.
- La conversion implicite (affectation entre types compatibles) reste possible, le compilateur peut avertir si elle dégrade la valeur.
- Pour aller plus loin : d'autres écritures existent (héritées du C) - à reconnaître, pas à utiliser.

```
int e = 42;  
double d1 = static_cast<double>(e); // explicite  
double d2 = e;                      // implicite
```

Types de conversions numériques



Avec 5 types entiers signés, 5 types non signés et 3 types réels, il y a 156 conversions possibles d'un type numérique à un autre. Pour simplifier, on les groupe en

- Promotions numériques vers int
- Conversions
 - Entier vers entier
 - Tout type numérique vers réel
 - Réel vers entier

Conversion entier $\rightarrow$ entier

- Toutes les conversions entières qui ne sont pas des promotions numériques
- La valeur est **préservée si elle est représentable** dans le type de destination
- Sinon, elle choisit la valeur dans l'intervalle représentable du type de destination sur n bits qui est **congrue modulo 2^n** (c-à-d même reste) avec la valeur de départ.

Note : Avant C++20, la valeur était indéfinie dans ce cas pour les types de destinations signés

```
for(int s : { 100, 200, 8100, 40000, -10 })  
    cout << setw(5) << s << " : " << setw(4) << +(unsigned char) s << "(uc) "  
        << setw(4) << +(signed char) s << "(sc) " << setw(5) << (unsigned short) s << "(us) "  
        << setw(6) << (signed short) s << "(ss)" << setw(11) << (unsigned int) s << "(ui)\n";
```

Le + devant (unsigned char) et (signed char) force la promotion en int : sans lui, cout afficherait un caractère au lieu d'un nombre.

Équivalent explicite : static_cast<int>((unsigned char) s)

Conversion entier $\rightarrow$ entier : résultats

- 100 est représentable dans tous les types : la valeur est conservée partout
- 200 $\rightarrow$ signed char : hors de $-128..127$, donc **200 - 256 = -56**
- 8100 $\rightarrow$ unsigned char : **8100 mod 256 = 164** ; les mêmes bits relus en signed char donnent **-92**
- 40000 $\rightarrow$ signed short : **40000 - 65536 = -25536**
- -10 vers un type non signé remonte en haut de l'intervalle : **246** (uc), **65526** (us), **$2^{32} - 10 = 4294967286$** (ui)

100	:	100 (uc)	100 (sc)	100 (us)	100 (ss)	100 (ui)
200	:	200 (uc)	-56 (sc)	200 (us)	200 (ss)	200 (ui)
8100	:	164 (uc)	-92 (sc)	8100 (us)	8100 (ss)	8100 (ui)
40000	:	64 (uc)	64 (sc)	40000 (us)	-25536 (ss)	40000 (ui)
-10	:	246 (uc)	-10 (sc)	65526 (us)	-10 (ss)	4294967286 (ui)

Conversions vers des réels

- La conversion de float en double est une promotion numérique : elle ne modifie pas la valeur.
- Pour les autres conversions vers `float`, `double` et `long double`
 - Si la valeur peut être représentée **exactement**, elle ne change pas
 - Si la valeur est comprise **entre deux valeurs représentables**, le choix entre ces deux valeurs dépend de l'implémentation.
 - Sinon, le résultat est indéfini

```
float a = 0.5;          cout << setprecision(10) << a << endl;  
float b = 3.1415926536; cout << setprecision(10) << b << endl;  
float c = 1e41;         // c est indéfini, probablement +inf
```

0.5
3.141592741

Conversion réel vers entier

- Pour convertir un réel en entier, **la partie fractionnaire est tronquée**

```
int    a = 2.55;           // a = 2
int    b = -2.55;          // b = -2
double c = 2.55;
int    d = c + 0.5;         // troncature => 3
```

- Pour une valeur **non représentable** dans le type entier, le résultat est **indéfini**

Attention aux erreurs d'arrondi

- La conversion réel $\rightarrow$ entier peut donner des résultats surprenants

```
double d = 100 * 4.35;  
cout << d << " ?= " << int(d) << endl;
```

435 ?= 434

- Le calcul en réel se fait avec une précision finie en sa représentation binaire. Le résultat n'est pas exactement 435

```
cout << setprecision(20) << d << endl;
```

434.999999999999994316

- On résout ce problème en spécifiant explicitement la méthode d'arrondi avant de convertir en entier

```
cout << int(round(d)) << endl;
```

435

Conversions implicites

- Il est possible d'écrire des expressions arithmétiques en mélangeant les types. Par exemple quel est le type de l'expression suivante ?

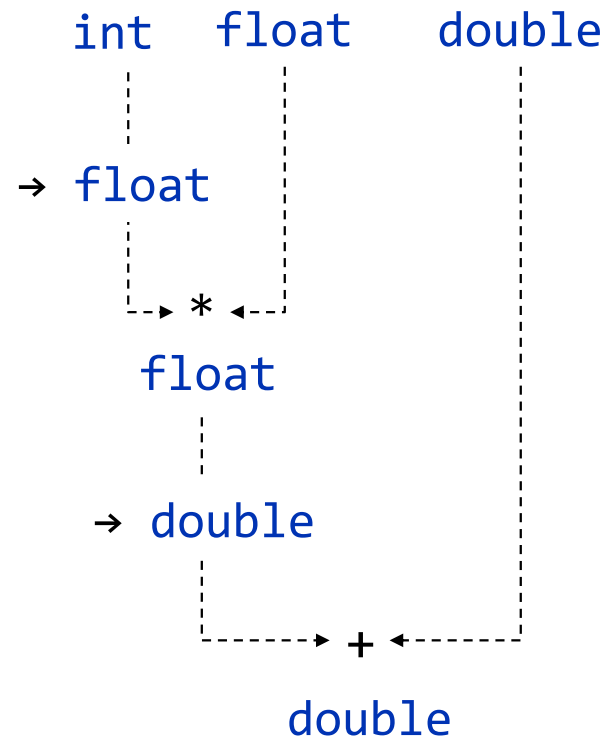
```
5 * 3.14F + 5.3e-2;
```

- Pourtant, les opérateurs arithmétiques binaires
 - ne sont définis que pour deux **opérandes de même type**
 - retournent** une valeur du **même type** que leurs opérandes
- Les opérateurs (unaires ou binaires) ne sont **pas définis** pour les types entiers plus courts que `int` (`signed char`, `unsigned char`, `signed short` et `unsigned short`)
- Il y a donc **conversion implicite des opérandes** avant d'appliquer l'opérateur

HE^{VD} IG Exemple



```
5 * 3.14F + 5.3e-2;
```



Attention aux comparaisons

- Les règles pour les opérateurs arithmétiques s'appliquent également aux opérateurs de comparaison
- Cela pose problème si l'on compare des entiers signés et non signés

```
signed int    a = -1;
unsigned int  b =  1;

cout << boolalpha; //pour afficher les bool

cout << (signed(-1) < signed(1)) << endl;
cout << (unsigned(-1) < unsigned(1)) << endl;
cout << (a < b) << endl;
```

true
false
false

Avertissements du compilateur

- Le compilateur peut nous avertir de ces conversions implicites dégradantes (perte de précision / dépassement possible) avec l'option **-Wconversion**

```
int unEntier = 3.14;
```

```
warning: implicit conversion from  
'double' to 'int' changes value from  
3.14 to 3 [-Wliteral-conversion]
```

- Le compilateur peut nous en avertir de conversions implicites entre types signés et non signés avec l'option **-Wsign-conversion**

```
int a = 4;  
unsigned int b = 5;  
cout << a - b; // affiche 4294967295
```

```
warning: implicit conversion  
changes signedness: 'int' to  
'unsigned int' [-Wsign-conversion]
```

Au delà du C++, quel que soit le langage ...

- Quels types sont disponibles pour traiter des entiers / des réels ?
- Quel est leur coût en mémoire ?
- Le langage dispose-t-il d'un type non signé ?
- Quelle est la plage des valeurs représentables par chaque type ?
- Quelle est la précision des calculs en réel ?
- Comment le dépassement est-il géré lors des calculs ?
- Quels opérateurs / fonctions mathématiques sont disponibles ?
- Comment convertir explicitement d'un type à l'autre ?
- Comment les conversions implicites sont-elles gérées dans les expressions hybrides (si elles sont autorisées) ?
- Comment les arrondis sont-ils gérés ?
- Comment formater ces nombres sous forme de texte ?
- Existe-t-il un type réel en base décimale pour les calculs financiers ?